

Halcyon Lab hardware roadmap: from one GPU to six trading desks

A used RTX 3090 at \$700 is the single most impactful upgrade you can make today, unlocking Qwen3 14B inference, feasible (if tight) GRPO training via Unsloth, concurrent FinBERT execution, and 24 GB of VRAM headroom — all for less than 11 months of cloud GPU rental at equivalent throughput. Your current RTX 3060 12 GB handles the existing 65–195 daily inference calls without strain, but it creates hard ceilings on model size, rules out on-device GRPO entirely, and leaves zero room for the multi-desk, multi-adapter architecture you’re targeting. The roadmap below moves from that \$700 first upgrade through a four-phase scaling plan that reaches six trading desks within your \$5K–\$20K two-year budget, with each phase gated by a concrete business milestone rather than hardware lust.

What every GPU tier actually unlocks

The critical variable for local LLM deployment isn’t raw compute — it’s VRAM. Token generation is memory-bandwidth-bound, `LocalLlm +2` meaning the GPU that can hold the full model in VRAM without spilling to system RAM wins decisively. A partially offloaded model runs **5–20× slower** than one fully resident in VRAM. `LocalLlm` `localllm`

RTX 3060 12 GB (current, ~\$200 used). Qwen3 8B Q8_0 at 8.7 GB fits with ~3.3 GB remaining, supporting roughly 8–10K context tokens before KV cache pressure forces degradation. At Q4_K_M, Qwen3 14B (8.3 GB file, ~10.7 GB VRAM at 8K context) squeaks in, but context length is severely limited `localllm` and concurrent FinBERT is impossible. GRPO training on Qwen3 8B is **not feasible** — even with Unsloth’s 80–90% memory reduction, `Unsloth` the 8B GRPO pipeline requires a minimum of ~16 GB. Generation speed for Qwen3 8B Q4_K at 16K context: **~42 tokens/sec**.

`hardware-corner` For Q8_0, expect ~33–35 t/s after the bandwidth penalty. The 3060 remains a capable inference card for your current single-model workflow, but it’s at the wall.

RTX 3090 24 GB (~\$700 used). This is where the architecture opens up. Qwen3 14B at Q5_K_M (10.5 GB file, ~12 GB VRAM at 8K context) fits comfortably with 12 GB remaining for KV cache, enabling 32K+ context. Qwen3 32B at Q4_K_M (19.8 GB file, ~22.2 GB at 8K context) barely fits but has no headroom for meaningful context.

Boredconsultant Concurrent Qwen3 8B Q8_0 + FinBERT (9.2 GB combined) leaves **14.8 GB** for context — effectively unlimited for trading prompts. GRPO training becomes marginal: with Unsloth QLoRA (4-bit, rank 16, batch_size=1, num_generations=4, max_seq_len ≤ 1024, gradient_checkpointing="unsloth"), Qwen3 8B GRPO fits in 24 GB, though training is slow and context-limited. Generation speed: **~87 t/s** (Q4_K, 16K),

hardware-corner ~70–75 t/s estimated at Q8_0. Prompt processing: **2,572 t/s**.

hardware-corner **Hardware Corner** The 3090 is the only consumer GPU supporting NVLink, which matters if you later add a second card. At **\$29/GB of VRAM**, it's the undisputed value champion for local AI in 2026. **XDA Developers**

RTX 4090 24 GB (~\$1,600 used). Same 24 GB VRAM, so identical model-fitting constraints as the 3090. The Ada Lovelace architecture delivers ~20% faster generation (104 t/s vs 87 t/s for Qwen3 8B at 16K) **hardware-corner** and better tensor core throughput for training. FP8 support in vLLM/TensorRT enables further memory savings, though GGUF workflows don't leverage this yet. The 4090 does **not** support NVLink — dual 4090s communicate only via PCIe. At 2.3× the price of a used 3090 for ~20% more speed with the same VRAM, the 4090 is a poor value proposition for inference-heavy workloads. It makes sense only when training speed is the bottleneck and you're committed to a single-GPU architecture.

RTX 5090 32 GB (~\$2,500–3,500 street). The first consumer card to break the 24 GB ceiling. **Runpod** Those extra 8 GB unlock Qwen3 14B Q8_0 (~15 GB) with 17 GB remaining for massive context windows, Qwen3 32B Q4_K_M with actual room for 16K+ context (50.9 t/s at 16K), **hardware-corner** and concurrent multi-model serving (14B + 8B + FinBERT simultaneously at 17.5 GB combined). GDDR7 bandwidth of **1,792 GB/s** **Vast.ai** **Kentino** — 78% higher than the 4090's 1,008 GB/s — translates directly to generation speed: **Runpod** **145 t/s** on Qwen3 8B, **103 t/s** on Qwen3 14B.

hardware-corner The 575W TDP demands a robust PSU (850W minimum) and good cooling. **Runpod** At current street prices, the 5090 costs 3.5–5× a used 3090 for ~65% faster inference and 33% more VRAM. Worth it only at scale or when 32 GB is genuinely required.

RTX 5060 Ti 16 GB (\$430–500 new). The sleeper pick. At \$28/GB, it matches the 3090 on VRAM value while running Qwen3 14B Q4_K_M at 32.9 t/s. **hardware-corner** Lower memory bandwidth (448 GB/s) means significantly slower generation than the 3090 (51 t/s vs 87 t/s on 8B), **hardware-corner** but for a dedicated FinBERT/embedding GPU or a second machine's inference card, it's remarkably capable at a third of the 3090's price.

Server GPUs: powerful but impractical for home deployment

A used **A100 40 GB PCIe** at \$5,000–7,000 offers 40 GB of HBM2e at 1,555 GB/s

bandwidth [Aitoldiscovery](#) — extraordinary for both inference and training. But it's passively cooled, requiring server-grade directed airflow that a desktop case cannot provide. Users report needing custom shrouds, zip-tied fans, and noise levels in the 50–70 dB range. [HP Support Community](#) The **A100 80 GB SXM** (\$5,000–9,000 used) [Aitoldiscovery](#) requires an SXM4 baseboard incompatible with consumer motherboards entirely. H100 PCIe (\$12,000–22,000) has the same passive-cooling problem at 350W TDP. For a solo home-office operator, consumer GPUs are dramatically simpler and nearly as capable per dollar.

GRPO training: the hardest constraint in the system

GRPO is fundamentally more memory-hungry than SFT because it generates **multiple completions per prompt** (typically 4–8), maintains both policy and reference models, and produces enormous logit tensors. [Oxen](#) Oxen.ai benchmarks on H100 show standard TRL GRPOTrainer with Llama 3.1-8B (LoRA + 8-bit Adam + gradient checkpointing, batch_size=1, num_generations=4) consuming **38–45 GB VRAM**. [X](#) At full fine-tune, 8B GRPO exceeds 75 GB. Qwen3 14B GRPO OOMs even on H100 80 GB without LoRA.

Unsloth changes the equation dramatically. Its optimized GRPO implementation shares weight memory between training and vLLM inference, uses an efficient linear cross-entropy algorithm (8× less logit memory), and applies async CPU offloading during gradient checkpointing. [Unsloth](#) Result: Llama 3.1-8B GRPO drops from 510 GB (standard TRL + FA2 at 20K context) to **54.3 GB** — a 90% reduction. [Unsloth](#) [X](#) Qwen3-8B FP8 GRPO with Unsloth reportedly fits in 16 GB. [Unsloth AI](#) With 4-bit QLoRA on a 24 GB RTX 3090, Qwen3 8B GRPO is possible at short context (≤1024 tokens), batch_size=1, num_generations=4, and gpu_memory_utilization tuned to 0.5–0.6.

The practical recommendation: **run GRPO on cloud A100 instances** (\$0.89/hour on RunPod) rather than constraining your local GPU. At 3 hours/week, that's \$11.57/month — trivial compared to buying a dedicated training GPU. Reserve your local RTX 3090 for the 90%+ workload that is inference.

Multi-LoRA serving makes six desks feasible on one GPU

The key architectural insight for scaling to six trading desks is that all six share the **same base Qwen3 8B model** — they differ only in their LoRA adapters (~34 MB each at rank 16). On a 24 GB GPU with Qwen3 8B Q5_K_M loaded (~7.3 GB VRAM), you have

~16 GB remaining for KV cache and adapters. Six adapters total just **204 MB** — negligible. The bottleneck isn't memory but serving architecture.

llama-server (llama.cpp) is the recommended Windows-native solution. It supports per-request LoRA selection via the API's `lora` field, hot-swapping via the `/lora-adapters` POST endpoint, and startup preloading via `--lora-init-without-apply`.

[GitHub](#) Hot-swap latency for GPU-resident adapters is **1–5 ms**; cold-loading from CPU cache takes 10–100 ms. The limitation: requests using different LoRA adapters are not batched together, so concurrent multi-desk inference processes sequentially. [GitHub](#) For your workload of ~200 calls/day (one call every ~2 minutes during market hours), this is entirely irrelevant.

vLLM offers superior multi-LoRA batching — different requests in the same batch can use different adapters simultaneously — but requires Linux or WSL2. Its `max_loras` parameter controls concurrent GPU-loaded adapters, while `max_cpu_loras` provides an LRU cache for hundreds of adapters. [vLLM](#) [Anyscale](#) Runtime loading/unloading is supported via REST API. [vLLM](#) If you move to a Linux inference server, vLLM becomes the natural choice.

Ollama supports one adapter per Modelfile via the `ADAPTER` instruction. The workaround for six desks: create six separate Modelfiles (TradingDeskA through TradingDeskF), each pointing to the same base model with a different adapter. Ollama shares the base model weights in VRAM. [Shshell](#) This works but lacks per-request routing flexibility.

Switching LoRA adapters **invalidates the KV cache** in all current implementations, since adapted weights modify key/value projections. A promising development is Activated LoRA (aLoRA, arXiv 2512.17910), which enables cross-model KV cache reuse by activating adapters only after an invocation sequence — achieving up to 58× latency reduction when switching adapters. [arXiv](#) [arXiv](#) PRs are submitted to both vLLM and llama.cpp. [GitHub](#)

CPU workloads are more capable than you might expect

FinBERT (BERT-base, ~110M parameters) as an ONNX INT8 model occupies just **200–300 MB of RAM** and processes a single document in **~18–20 ms** on a modern CPU with AVX-512 VNNI support. Processing 500 news headlines (5 per stock × 100 stocks) takes **under 10 seconds** without batching, or **2–3 seconds** with batch size 8–16. This is fast enough to run entirely on CPU, freeing your GPU exclusively for LLM inference.

Sentence embeddings via all-MiniLM-L6-v2 (~33M params, ~100 MB) achieve ~50

sentences/sec single-threaded, [Medium](#) scaling to **~14,000 sentences/sec** multi-threaded. [Milvus](#) HHEM 2.1 for hallucination detection runs at ~1.5 seconds per 2K-token input on CPU. [Hugging Face](#) [Dataloop](#) All three models can run concurrently, consuming a total of ~1–1.2 GB RAM.

AMD Ryzen 7000/9000 processors are strongly preferred for these workloads. Zen 4/5 architectures provide full AVX-512 + VNNI support, [Cortensor](#) which Intel dropped in 12th–14th gen hybrid architectures. ONNX Runtime’s INT8 matmul with AVX-512 VNNI delivers **~2.5x speedup** over FP32 on AVX2-only CPUs. [Microsoft Open Source](#) A Ryzen 7 9700X (8-core, ~\$300) or Ryzen 9 9900X (12-core, ~\$400) is ideal. DDR5 provides ~15–25% improvement over DDR4 for ML inference. [Micron](#)

32 GB RAM is sufficient for the current workload (OS ~5 GB + 12 data collectors ~1.2 GB + NLP models ~1.2 GB + trading app ~1 GB ≈ ~8–9 GB). With a local LLM loaded (~5 GB for 8B Q4), total reaches ~14 GB. Upgrade to **64 GB** when you run concurrent LLM inference + training + all NLP models + data collectors simultaneously.

The case for Linux and a dedicated inference machine

CUDA compute is **~5–15% faster on Linux** than Windows across multiple benchmark suites, [Phoronix](#) attributable to Windows’ WDDM (Windows Display Driver Model) overhead. [Einstein@Home](#) More critically, the entire ML training ecosystem — DeepSpeed, Megatron-LM, Unsloth, vLLM — targets Linux first. DeepSpeed on Windows requires painful manual compilation and lacks multi-GPU support. Megatron-LM is effectively Linux-only. Ollama on Windows has documented GPU detection failures and slower model loading due to Windows Defender DLL scanning (90-second startup timeout vs 30 seconds on Linux).

The recommended architecture for a solo operator: **a dedicated headless Linux machine** (Ubuntu 24.04) running the inference GPU, accessible via SSH from your Windows trading workstation. This eliminates all Windows ML pain points, enables 24/7 operation, and cleanly separates trading GUI concerns from model serving. VS Code Remote-SSH provides a seamless development experience. Use Tailscale for secure remote access. Keep your Windows machine for the trading interface, data visualization, and daily workflow.

WSL2 is a viable intermediate step — CUDA performance within 1–5% of native Linux for sustained workloads [NVIDIA Developer](#) — but adds complexity and an extra memory overhead of ~1–2 GB for the Linux VM. Dual-boot is impractical because you can’t run the trading GUI and ML training simultaneously.

Power, cooling, and the home-office reality

A single RTX 3090 system draws **~550W under full inference load** (LanOC (350W GPU TechReviewer Best Value GPU + 200W system), SitePoint well within a standard US 15A/120V circuit's 1,440W continuous rating. A dual RTX 3090 system draws ~900W — still safe on a 20A circuit (1,920W continuous) with monitors and peripherals (~200W), but don't share that circuit with a space heater or window AC unit.

Power-limiting to 220W per GPU is the efficiency sweet spot for inference.

Benchmarks show minimal throughput loss (~5–10%) at 37% power reduction, dropping a dual 3090 system from ~900W to ~640W. Heat output drops proportionally: 640W is equivalent to a modest space heater, manageable in a room with normal ventilation. Noise at 220W is near-silent for quality AIB cards (ASUS ROG Strix, EVGA FTW3).

UPS is non-negotiable for a trading system. A CyberPower CP1500PFCLCD (1500VA/1000W, KommandoTech pure sine wave, ~\$220) covers a single-GPU system with 5–10 minutes of runtime for graceful shutdown. Pure sine wave output is critical (PC Gamer — modified sine wave units cause coil whine and PSU instability with high-end GPUs. H|ard|Forum For dual GPU, step up to a 2000VA+ unit. H|ard|Forum

Phased hardware roadmap

Phase 0: Optimize current hardware (now, \$0)

Milestone gate: None — do this immediately.

Switch from Q8_0 to **Q5_K_M** quantization for Qwen3 8B. Quality retention is ~98–99% of FP16 (vs Q8_0's ~99.9%), but VRAM drops from 8.7 GB to ~5.7 GB, freeing **3 GB** for longer context and concurrent FinBERT. At ~80 t/s estimated on RTX 3060 (vs ~33 t/s at Q8_0), inference speed nearly doubles. For financial text analysis, Q5_K_M preserves instruction-following and reasoning stability; the perplexity increase of +0.035 is imperceptible in practice.

Deploy FinBERT as ONNX INT8 on CPU. At 18–20 ms/doc, your 100-stock news pipeline completes in under 10 seconds without touching the GPU. Set OLLAMA_KEEP_ALIVE=-1 to pin the model in VRAM permanently during market hours (Markaicode (idle power draw: ~15–30W). Estimated cost: \$0. Performance gain: ~2× inference speed, concurrent NLP capability.

Phase 1: The 3090 upgrade (\$800–1,200, month 1–3)

Milestone gate: First live trading strategy generating consistent signals.

Purchase a **used RTX 3090 24 GB** (\$700–800). [XDA Developers](#) Install it in a dedicated headless Linux machine built from budget components:

- AMD Ryzen 7 9700X (\$300) — AVX-512 for CPU NLP inference
- 32 GB DDR5-5600 (\$80)
- B650 motherboard (\$120) — ensure PCIe 4.0 x16 slot
- 750W 80+ Gold PSU (\$90) — sufficient for single 3090 at 220W power limit

[Futuredly](#)

- 500 GB NVMe Gen4 (\$45)
- Basic ATX case (\$60)
- **Total: ~\$1,500 including GPU**

This machine runs Ollama (or vLLM) serving Qwen3 8B Q5_K_M with 16+ GB VRAM headroom, alongside ONNX Runtime for FinBERT, embeddings, and HHEM. Your Windows workstation calls the Linux machine's API over gigabit LAN (sub-ms network overhead, negligible vs 6–15 second inference time).

Repurpose the RTX 3060 machine as your trading workstation and data collection hub. Weekend QLoRA SFT continues on the 3090 (~8–15 minutes for 1,000 examples vs 15–30 minutes on the 3060). Monthly power cost: **+\$16** for the inference server.

Phase 2: Multi-desk LoRA deployment (\$200–500, month 4–8)

Milestone gate: 3+ validated trading strategies ready for concurrent deployment, AUM $\geq$ \$50K.

No new GPU hardware needed. Deploy **llama-server** (or vLLM on Linux) with multiple LoRA adapters in GGUF format. Convert each strategy's fine-tuned adapter using llama.cpp's `convert_lora_to_gguf.py`. [Hugging Face](#) [Towards AI](#) Pre-load all adapters at startup with `--lora-init-without-apply`. [GitHub](#) Each trading desk sends requests specifying its adapter via the API's `lora` field. [GitHub](#)

Six rank-16 adapters total 204 MB — invisible against 24 GB VRAM. With Qwen3 8B Q5_K_M (~7.3 GB) + six adapters (~0.2 GB) + 8K KV cache (~1.2 GB), total VRAM usage is ~8.7 GB, leaving **15.3 GB** free. You could run Qwen3 14B Q5_K_M (12 GB) with all six adapters and still have headroom.

Investment: software engineering time plus ~\$200–500 for additional NVMe storage (training data, model checkpoints, backups). Monthly power cost: unchanged.

Phase 3: GRPO and advanced training (\$500–2,000, month 6–14)

Milestone gate: SFT-trained models plateau in performance; reinforcement learning from reward signals needed, AUM $\geq$ \$100K.

Do not buy hardware for GRPO. Rent cloud compute:

- **Weekly GRPO session on RunPod A100 80 GB:** $\$0.89/\text{hour} \times 3\text{--}4 \text{ hours} = \$2.67\text{--}\$3.56/\text{week} = \sim\$14/\text{month}$
- **Monthly intensive training sprint:** 20 hours on A100 = \$17.80
- **Annual GRPO compute cost:** $\sim\$170\text{--}\215

This is 4–5 $\times$ cheaper than buying an A100 (\$5,000+) or even a second RTX 3090 (\$700+), given the intermittent nature of training. Use Unsloth + TRL GRPOTrainer with QLoRA 4-bit, rank 16, num_generations=4, max_seq_len=1024. [Unsloth AI +2](#) Sync training data and model checkpoints to/from cloud via rclone + B2/S3.

If cloud becomes impractical (data sensitivity, reliability), a **second RTX 3090** (\$700) in the Linux machine enables on-premises GRPO with Unsloth at short context. This requires a PSU upgrade to 1200W (\$150) and a motherboard with two PCIe x16 physical slots. NVLink bridge (\$80–350) provides [Kentino](#) 112.5 GB/s inter-GPU bandwidth, but benchmarks show only **2.7% improvement** over PCIe-only for inference [URE](#) — save the money unless you're splitting large models across both cards.

Phase 4: Scale-out architecture (\$2,000–5,000, month 12–24)

Milestone gate: 6 active desks, need for concurrent inference + training without interference, AUM $\geq$ \$250K.

Two architectures, depending on model size requirements:

Option A: Dedicated inference + training GPUs (\$2,000). Add a second RTX 3090 to the Linux machine. GPU 0 handles all inference (six-desk multi-LoRA serving via vLLM). GPU 1 handles training (QLoRA SFT weekdays, GRPO weekends). The two workloads never compete for VRAM. Combined 48 GB enables Qwen3 32B Q4_K_M via tensor parallelism (30.3 t/s on dual 3090) or running two completely independent 14B models.

Option B: Next-gen single GPU (\$2,500–3,500). Replace the RTX 3090 with an RTX 5090 32 GB. [Runpod](#) The 32 GB VRAM runs Qwen3 14B Q8_0 (15 GB) with all six LoRA adapters and massive context, [Runpod](#) at **103 t/s** [hardware-corner](#) — nearly 2 $\times$ faster than the 3090. GDDR7's 1,792 GB/s bandwidth [Vast.ai](#) makes this the fastest single-card solution available. [Runpod](#) Use the displaced 3090 as a dedicated training GPU or

sell it (\$500–600 by then) to offset costs.

Five-year total cost of ownership

Configuration	Hardware	5-Year Power	Resale	5-Year TCO
Phase 1: Single RTX 3090 system	\$1,500	\$965	-\$200	\$2,265
Phase 3: + Cloud GRPO (3h/week)	+\$0	+\$1,075	—	\$3,340
Phase 4A: + Second RTX 3090	+\$850	+\$1,575 total	-\$300	\$4,390
Phase 4B: RTX 5090 replacement	+\$2,500	+\$1,360 total	-\$900	\$5,225
Cloud-only (8h/day inference)	\$0	\$0	\$0	\$4,523 (Vast.ai 3090)

Owned hardware breaks even with cloud rental at **~20 months** for 8h/day inference workloads. Training-only cloud (\$14/month) remains cheaper than hardware indefinitely.

Risk assessment and single points of failure

The most dangerous risk in this architecture is **single-GPU failure during market hours**. If the inference GPU dies, all six trading desks lose their AI signal simultaneously. Mitigations:

- **Immediate:** Maintain a fallback to your RTX 3060 workstation running Ollama. Script an automatic failover that detects inference server unavailability and routes requests to the local backup. The 3060 runs Qwen3 8B Q5_K_M at ~42 t/s — slower but functional.
- **Phase 4:** Dual-GPU provides N+1 redundancy. Either GPU can serve inference if the other fails.
- **Data integrity:** SQLite with WAL mode handles the single-writer pattern cleanly. Back up daily to an external drive and weekly to cloud (Backblaze B2 at

\$5/TB/month). Trading positions should be recoverable from broker API state, not solely from local database.

- **Power:** UPS (1500VA pure sine wave) provides 5–10 minutes for graceful shutdown, [PC Gamer](#) preventing data corruption mid-trade.
- **Network:** Gigabit LAN between machines adds a failure point. Use direct ethernet cable (no switch dependency) for the inference link; keep a USB ethernet adapter as backup.

The model efficiency wildcard changes everything by 2028

The strongest argument against over-investing in hardware today is the “**densing law**”: capability per parameter doubles approximately every 3.5 months. [Nature](#) At this rate, by late 2027 a **4B model will match current Qwen3 8B quality** on domain-specific tasks. Already, Qwen3 30B-A3B (MoE, 30B total / 3B active parameters) [APXML](#) delivers 14B-class quality at **196 t/s on an RTX 4090** [hardware-corner](#) — faster than 8B dense models while fitting comfortably in 24 GB at Q4. [Awesome Agents](#) Speculative decoding adds another 2–3× on top, with combined quantization+speculation achieving **3.6× throughput gains** in production deployments. [Introl](#) [Prem AI](#)

This means your RTX 3090's 24 GB won't just run current 8B models — by 2028, it will run models with today's 32B-class reasoning capability. The hardware investment with the longest useful life isn't the biggest GPU; it's the one with enough VRAM to ride the efficiency curve. **24 GB remains the sweet spot through 2027**; 32 GB (RTX 5090) extends relevance to 2029+. Consumer 48 GB cards won't arrive until the RTX 60-series in late 2027 at earliest, more likely 2028.

Conclusion: buy the 3090, architect for modularity

The optimal strategy isn't a single massive hardware purchase — it's a modular architecture where each component can be upgraded independently. Start with the RTX 3090 Linux inference server (**\$1,500 all-in**), which handles your current and near-term workload with 3–4× headroom. Deploy multi-LoRA serving when strategies are validated. Rent cloud A100s for GRPO training at **\$14/month** rather than buying dedicated training hardware. Add a second GPU or upgrade to the 5090 only when AUM justifies the investment and workload genuinely demands it. The phased approach keeps your total two-year spend under \$3,500 while delivering the same capability as a \$10,000+ build — because most of that capability goes unused 90% of the time, and model efficiency gains will make today's expensive hardware purchases look premature

within 18 months.